

BICOL UNIVERSITY POLANGUI

College of Engineering - Computer Engineering Department

Lab Report: Arrays

Application: Campus Navigation System

Submitted by:

Ravago, Carl Davin, B.

Pelaez, Kyla

Orpiana, Mateo

Gil, Sherwin

Danao, Francis

Submitted to:

Melissa Jeankie Rellon

September 20, 2026

Objectives

At the end of this project about Array, the group should be able to:

- Apply array-based data structures to solve real problems within the Campus Navigation System.
- Implement linear search, array-based logging(with fixed limit), and array filtering in Python.
- Test each implementation using normal, boundary, and invalid input cases.
- Analyze the behavior and limitations of array-based solutions before upgrading to more advanced data structures in later lectures

Problem Statement

1. Building Search

Students and visitors need a fast way to locate a specific building on campus by name, without manually looking at a map of some sort. This module solves the problem using linear search over an array of building records, matching any building whose name contains the search term.

2. Navigation History

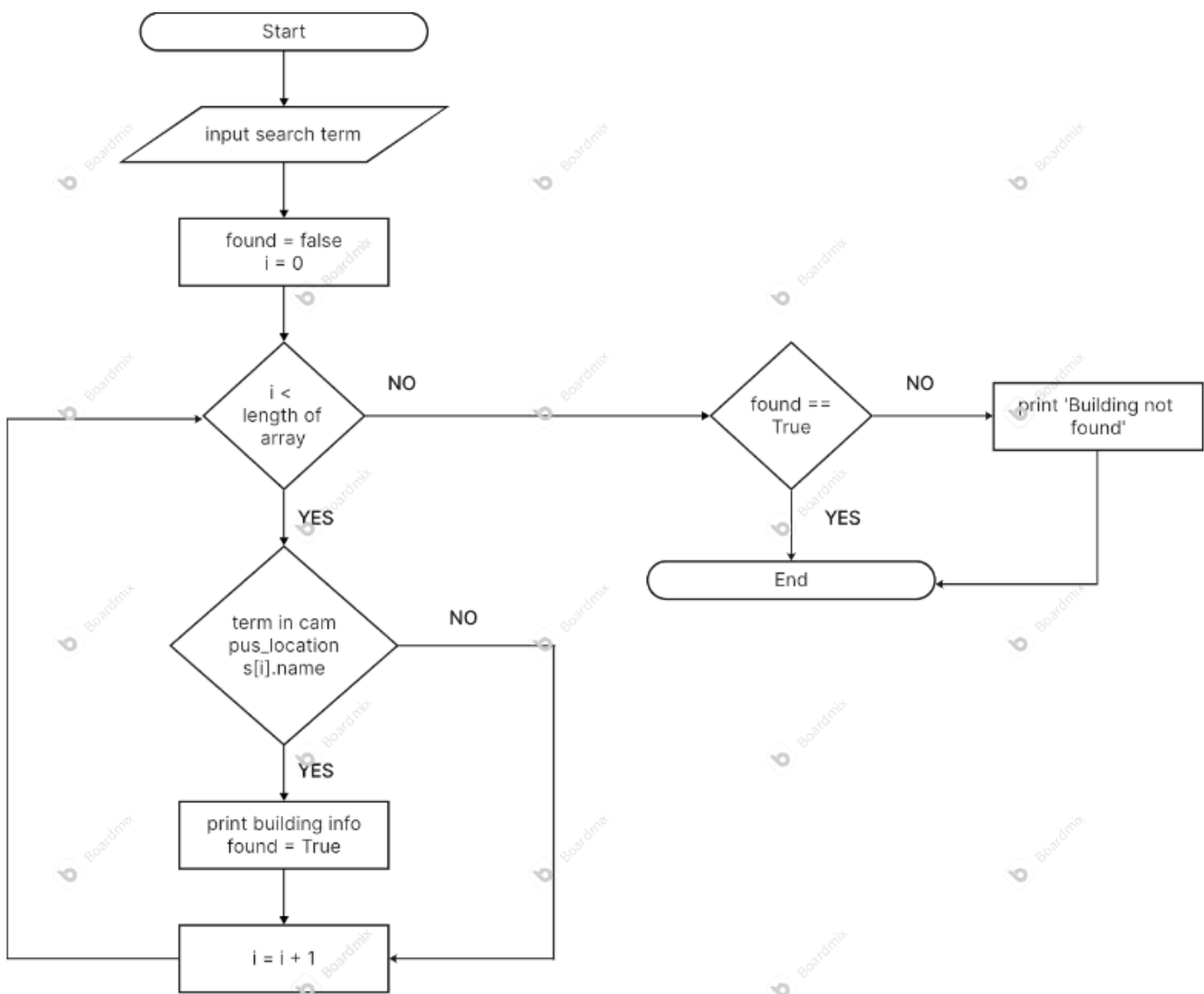
Users navigating the system may want to recall which buildings they recently looked up, without repeating a search. This module solves the problem by maintaining a fixed-capacity array (5 entries) that logs recent search terms in order, automatically discarding the oldest entry once the array is full (FIFO).

3. Building Statistics/Filter

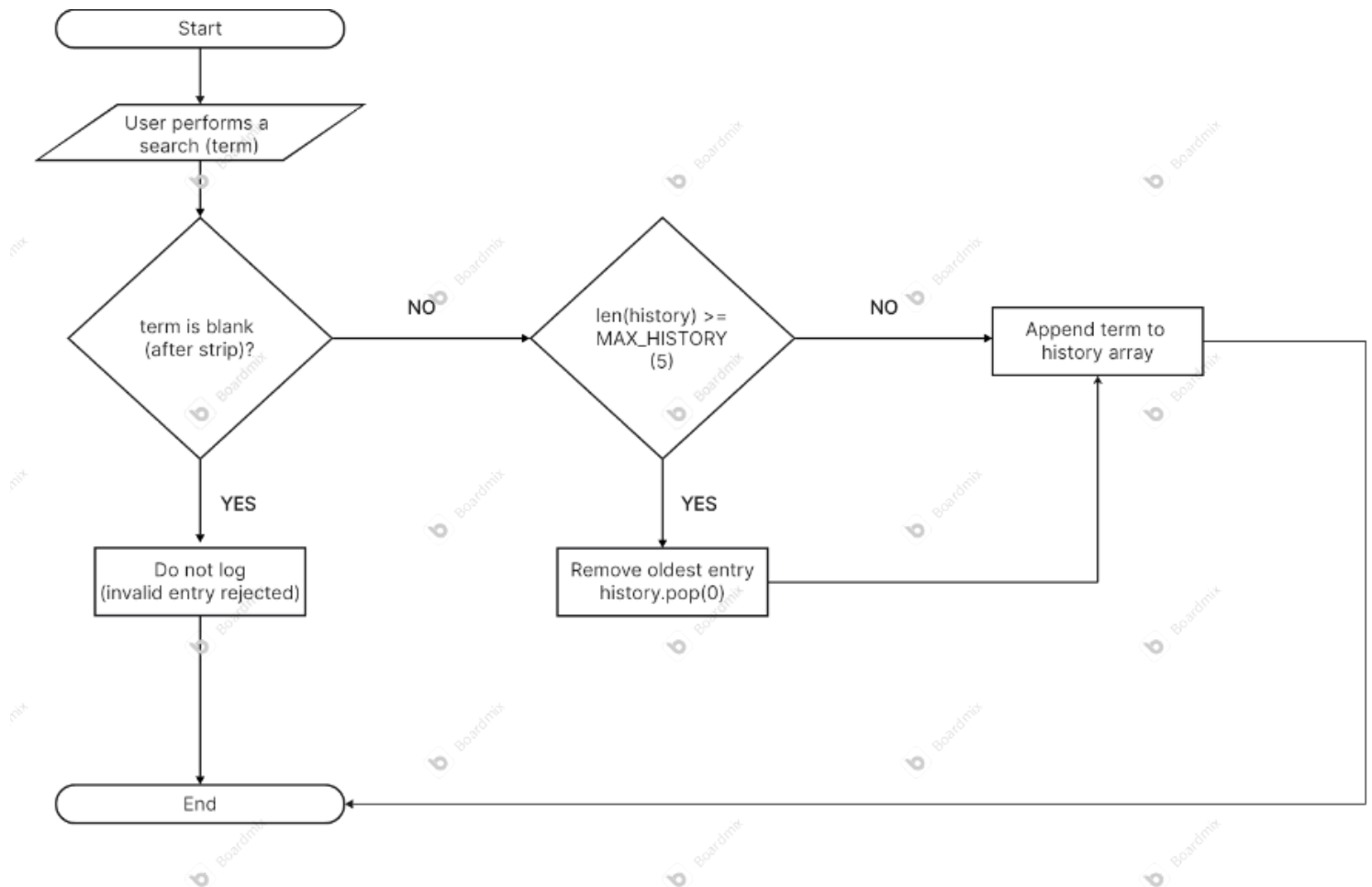
Users planning a visit often want to know how many buildings exist per category (e.g., Academic, Administrative, Facility), or see only the buildings under one category, instead of scanning the full list. This modules solves the problem by aggregating and filtering the same building array by category

Flowcharts

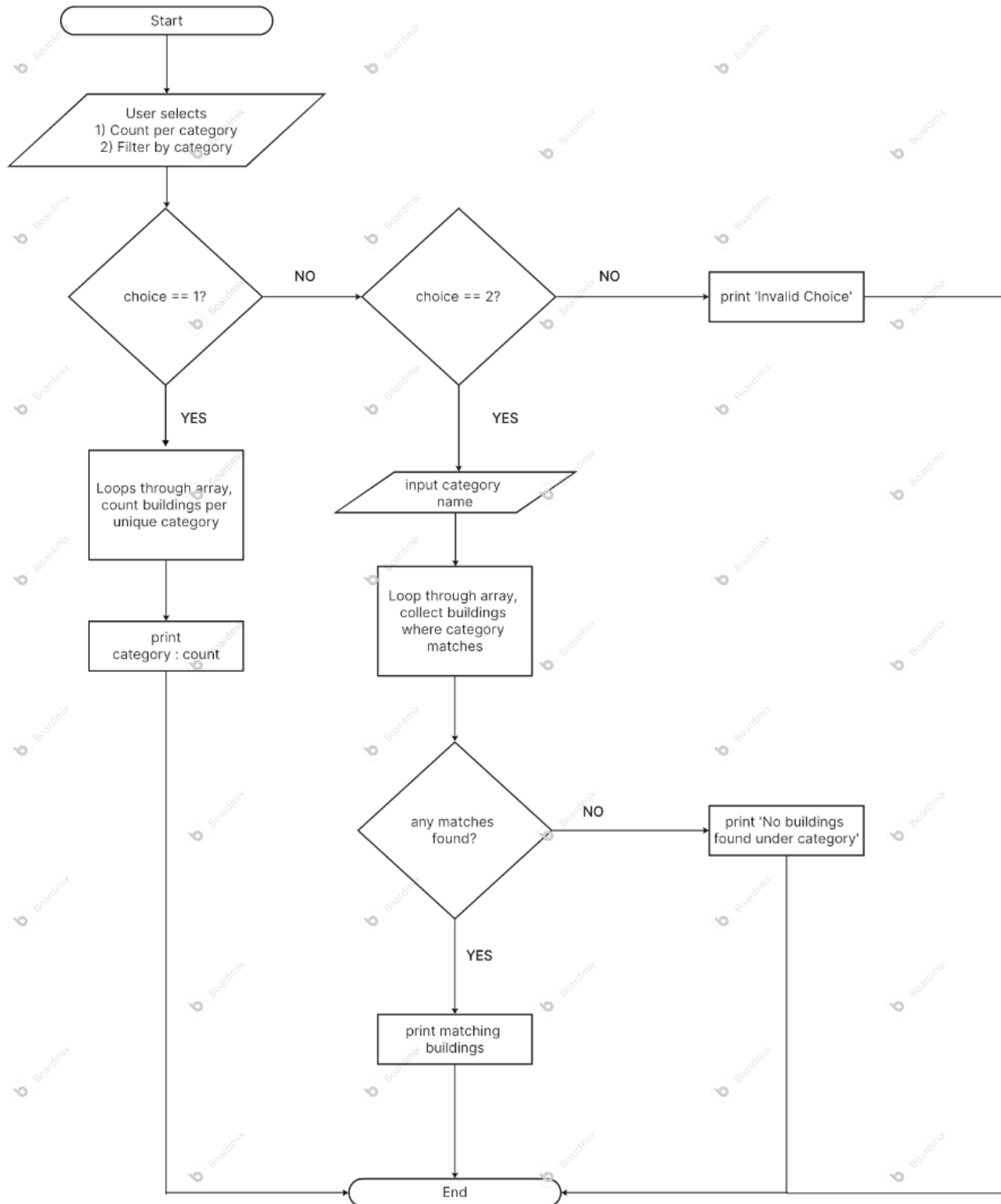
1. Building Search (Linear Search)



2. Navigation History (Array Log, Fixed Capacity)



3. Building Stats / Filter



Source Code Snapshots

1. Building Search (Linear Search)

```
def search_building():
    # Linear Search Algorithm
    search_term = input("\nEnter the name of the building to search:
").lower()
    log_search(search_term)
    found = False

    print("\nSearch Results")
    for loc in campus_locations:
        if search_term in loc.name.lower():
            print(loc.display_info())
            found = True

    if not found:
        print("Building not found. Please try another name")
```

2. Navigation History (Array Log, Fixed Capacity)

```
def log_search(term):
    term = term.strip
    if not term:
        return
    if len(navigation_history) >= MAX_HISTORY:
        navigation_history.pop(0)
    navigation_history.append(term)
```

```
def view_history():
    print(f"\nNavigation History (last {MAX_HISTORY} searches)")
    if not navigation_history:
        print("No search history yet.")
        return

    for i, term in enumerate(navigation_history, start=1):
        print(f"{i}. {term}")
```

3. Building Stats / Filter (Array Aggregation)

```
def building_stats():
    categories = []
    for loc in campus_locations:
        if loc.category not in categories:
            categories.append(loc.category)

    print("\nBuilding Stats / Filter by Category")
    print("[1] View building count per category")
    print("[2] Filter Buildings by category")
    choice = int(input("Select an option [1 - 2]"))

    match choice:
        case 1:
            print("\nBuilding Counter per Category")
            for cat in categories:
                count = 0
                for loc in campus_locations:
                    if loc.category == cat:
                        count += 1
                print(f"{cat}: {count}")
        case 2:
            print("\nAvailable Categories:", ", ".join(categories))
            cat_input = input("Enter category to filter: ").strip()

            matches = []
            for loc in campus_locations:
                if loc.category.lower() == cat_input.lower():
                    matches.append(loc)

            if matches:
                print(f"\nBuildings under {cat_input}")
                for loc in matches:
                    print(loc.display_info())
            else:
                print(f"No buildings found under category '{cat_input}'")
        case _:
            print("Invalid choice. Please enter [1 - 2]")
```

4. models.py - Location class (category field added)

```
class Location:

    def __init__(self, building_id: int, name: str, category: str):

        self.building_id = building_id

        self.name = name

        self.category = category

    def display_info(self):

        return f"[{self.building_id}] - {self.name} ({self.category})"
```

5. database.py – campus_locations array (categories assigned)

```
from models import Location

campus_locations = [

    Location(1, "CESD", "Academic"),

    Location(2, "Administration Building", "Administrative"),

    Location(3, "Salceda Building", "Academic"),

    Location(4, "Nursing Department", "Academic"),

    Location(5, "Gymnasium", "Facility")

]
```

Test Case & Output

Each module was tested with the three required input categories: Normal (valid), Boundary/Edge, and Invalid input. All outputs below are taken directly from running the program (main.py).

1. Building Search

Test Type	Sample Input	Program Output
Normal	Menu: 1 Search: gym	Search Results [5] Gymnasium (Facility)
Boundary	Menu: 1 Search : " "	Search Results [1] CESD (Academic) [2] Administration Building (Administrative) [3] Salceda Building (Academic) [4] Nursing Department (Academic) [5] Gymnasium (Facility) (An empty string is contained In every name, so all 5 Buildings are returned)
Invalid	Menu: 9	Invalid choice Please enter [1 - 5]

2. Navigation History

Test Type	Sample Input	Program Output
Normal	Search: gym Search: cesd/CESD Search: nursing Menu: 2	Navigation History (last 5 searches) 1. Gym 2. cesd 3. nursing
Boundary (array full)	3 more searches: Administration, salceda, gym	Navigation History (last 5 searches) 1. cesd 2. nursing

	(6th search total) Menu: 2	3. administration 4. salceda 5. Gym (‘gym’ from the 1st search was removed - array stays at MAX_HISTORY = 5).
Invalid	Search: “ ” (blank/whitespace) Menu: 2	Navigation History (last 5 searches) 1. cesd 2. nursing 3. administration 4. salceda 5. Gym (History unchanged - blank input is rejected before being logged).

3. Building Stats/Filter

Test Type	Sample Input	Program Output
Normal	Menu: 4 Sub-menu: 2 Category: Academic	Buildings under ‘Academic’ [1] CESD (Academic) [3] Salceda Building (Academic) [4] Nursing Department (Academic)
Boundary	Menu: 4 Sub-menu: 2 Category: Recreational	No buildings found under Category ‘Recreational’
Invalid	Menu: 4 Sub-menu: 5	Invalid choice. Please enter [1 - 2]

